

OCI Provisioning Automation

Java + Go services and Terraform for VM and image lifecycle

The project explained end to end, in simple terms

The project (as written on the resume): Cut provisioning failures 45% as measured by fleet dashboards by building Java and Go services and Terraform automation that integrate VM and image lifecycle with OCI Compute, VCN, Block Volume, Object Storage, IAM, OCIR, and OKE.

What is inside this document

- Section 1 - The simple summary (the big picture)
- Section 2 - Key words explained in plain English
- Section 3 - Architecture diagram
- Section 4 - The architecture diagram explained, piece by piece
- Section 5 - Every component explained in detail
- Section 6 - The complete workflow, step by step
- Section 7 - VM and image lifecycle states
- Section 8 - Where the 45% came from, and how it was measured
- Section 9 - Safeguards and a 60-second explanation

Note: The resume line gives the result and the technologies. This document explains how such a system is typically designed on Oracle Cloud. Exact service names, frameworks, and numbers may differ in your real setup. The numbers in the chart are illustrative examples; your real numbers come from your fleet dashboards.

1. The Big Picture (Simple Summary)

1.1 The one-line version

I built a set of Java and Go services plus Terraform code that create, manage, and retire virtual machines and OS images on Oracle Cloud automatically and safely. The system checks everything before it starts, retries smartly when something goes wrong, cleans up after itself, and always uses a valid image - so 45% fewer requests for new machines failed, as shown on the fleet dashboards.

1.2 Explained with an everyday example

Imagine ordering a new office for a new employee: a desk, a computer, a network cable, a key card, and a filing cabinet.

- **Before:** each team ordered the parts themselves, by hand, from different departments. Orders failed because the building was full, the budget was used up, the key card was not approved, there were no free network ports, or they ordered a computer model that no longer existed. Half-finished offices were left behind, blocking space for the next order.
- **After:** there is one **reception desk** (the Provisioning API). You tell it what you need. A **checker** confirms there is space, budget, permission, network ports, and a current computer model **before** anything is ordered. A **project manager** orders each part in the right order, tries again if a supplier is briefly busy, tries another floor if one is full, and **takes everything back** if the order cannot be finished. A **final inspector** confirms the office really works before handing over the keys. A **cleaner** walks the building every night removing anything left over.
- **A notice board** (the fleet dashboard) shows how many orders succeeded and why the others failed, so the team keeps fixing the biggest reasons.

In this project the office is a **VM**, the parts are **OCI services** (Compute, VCN, Block Volume, Object Storage, IAM, OCIR, OKE), and the reception desk, checker, manager, inspector, and cleaner are **Java and Go services plus Terraform**.

1.3 The resume line, phrase by phrase

Phrase	What it means in simple words
Cut provisioning failures 45%	Requests to create machines failed 45% less often. Example: 4 in 100 failed before, 2.2 in 100 after.
Provisioning	Creating and setting up a new machine with its network, disks, and permissions.
as measured by fleet dashboards	The number comes from live dashboards that track every request across all machines (the fleet).

Phrase	What it means in simple words
Java and Go services	Programs written in Java (the API and image catalog) and Go (checks, workflow, readiness, cleanup).
Terraform automation	Code that describes cloud resources, so they are created the same way every time.
integrate VM and image lifecycle	Manage machines and OS images from birth to retirement - and make sure machines always use valid images.
OCI Compute	Oracle's service for virtual machines.
VCN	Virtual Cloud Network - the private network, subnets, and firewall rules.
Block Volume	Disks for VMs (boot disk and data disks).
Object Storage	File storage for image files, Terraform state, and logs.
IAM	Identity and Access Management - who and what is allowed to do what.
OCIR	Oracle Cloud Infrastructure Registry - stores the container images of our services.
OKE	Oracle Kubernetes Engine - the Kubernetes cluster where our services run.

1.4 Why this matters

- **Failed provisioning blocks people:** engineers wait, test pipelines break, releases slip.
- **Half-created resources cost money** and use up quota, causing more failures later.
- **Most failures are predictable:** full quota, no capacity, missing permission, no free IPs, wrong image. Checking first prevents them.
- **One front door** means one set of rules, one set of logs, and one dashboard.
- **Data-driven improvement:** the dashboard shows the top failure causes, so the team fixes the biggest first.

2. Key Words in Plain English

Word	Simple meaning	Example
Provisioning	Creating a ready-to-use machine with everything it needs.	New VM with network and disk
Provisioning failure	A request to create a machine that did not finish successfully.	"Out of host capacity"
Fleet	All the machines being managed.	20,000 VMs
Lifecycle	All stages from creation to deletion.	Create, use, stop, terminate
VM	A virtual machine.	An Oracle Linux VM
Image	A saved OS disk template used to create VMs.	ol9-2026.09
Shape	OCI machine size/type.	VM.Standard.E5.Flex
OCID	Oracle Cloud ID - unique ID of any OCI resource.	ocid1.instance.oc1...
Region / AD / FD	Area / data center / hardware group inside a data center.	us-ashburn-1 / AD-2 / FD-3
Compartment	A folder in OCI for organizing resources and permissions.	team-a-dev
Service limit / quota	Maximum resources you are allowed to use.	200 E5 cores
Capacity	Whether the data center has free hardware of that type right now.	
Subnet / NSG	A slice of the network / a firewall rule group.	10.0.1.0/24
API	A way for programs to ask a service to do something.	POST /v1/vms
REST	A common style of web API using HTTP.	GET /v1/requests/123
SDK	A library that makes calling an API easy from code.	OCI Java SDK, OCI Go SDK
Java / Go	Programming languages. Java: mature, great for business services. Go: fast, simple, great for concurrent cloud tooling.	
Terraform	Infrastructure-as-code tool: you describe resources; it creates or changes them to match.	terraform apply
Terraform module	A reusable package of Terraform code.	modules/vcn

Word	Simple meaning	Example
Terraform state	Terraform's record of what it created. Stored remotely and locked so two runs do not collide.	Stored in Object Storage
Plan / apply	Preview the changes / make them.	terraform plan
Drift	When the real cloud no longer matches the code.	Someone changed a rule by hand.
Idempotency	Doing the same request twice has the same effect as once - no duplicates.	Retry token
Retry with backoff	Try again after waiting, waiting longer each time, with a bit of randomness (jitter).	1s, 2s, 4s, 8s
Throttling (429)	The API says "too many requests, slow down".	HTTP 429
Rollback / compensation	Undoing the steps already done when a later step fails.	Delete the half-made VM
Orphan	A resource left behind with no owner (for example from a failed run).	Unattached volume
Reconciler	A background job that compares "what should exist" with "what exists" and fixes differences.	
Container / Kubernetes	A packaged app / a system that runs and manages many containers.	OKE
Workload identity / instance principal	A way for services to call OCI with their own identity, without stored passwords.	
OCI Vault	Secure storage for secrets and keys.	
Dashboard	A live screen of charts and numbers.	Success rate 97.8%
p50 / p95	Typical value / value that 95% of requests are faster than.	p95 time-to-ready 6 min

3. Architecture Diagram

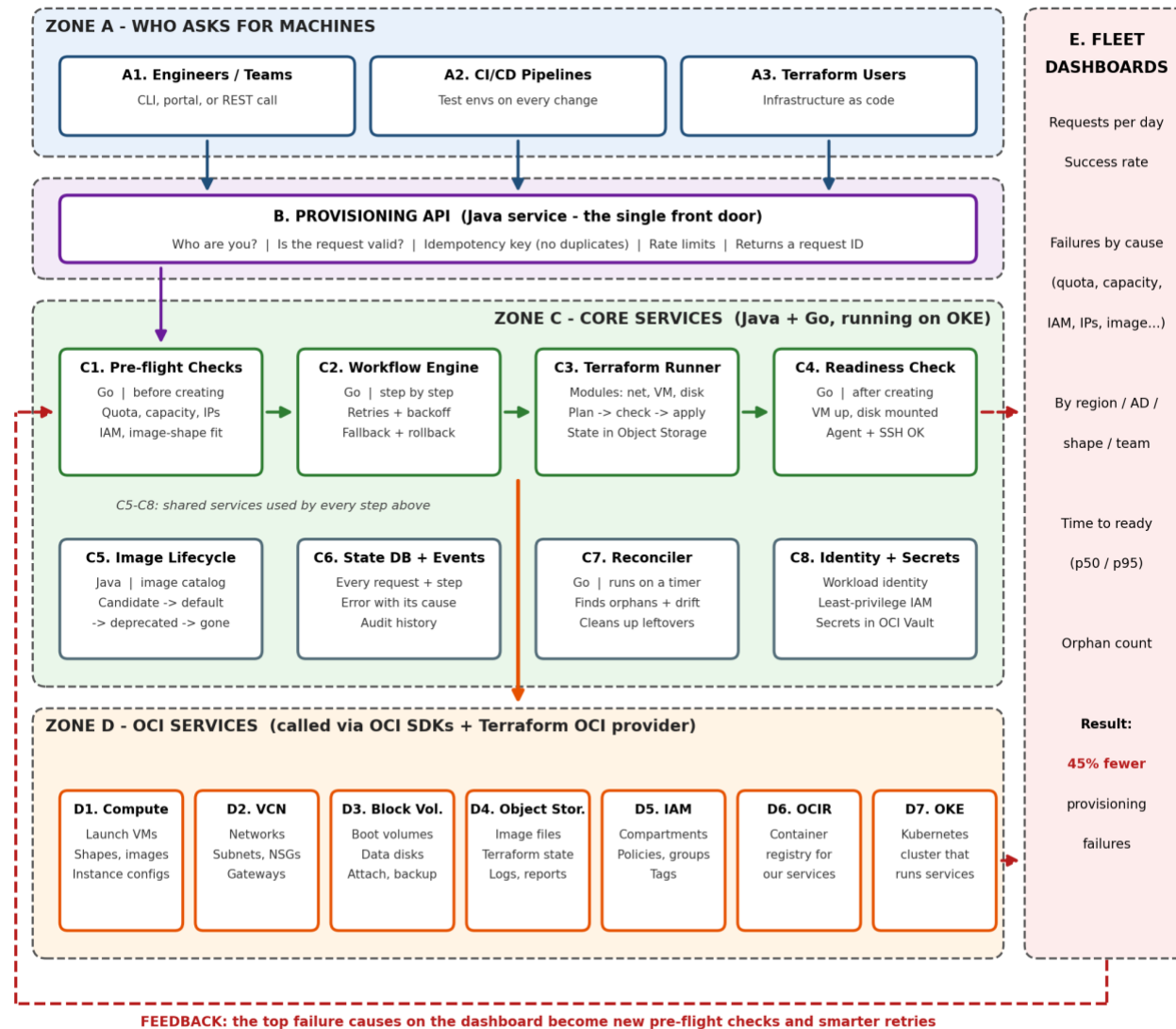


Figure 1 - Requests enter one API, pass through checks, a workflow, Terraform, and a readiness check, and use seven OCI services. Labels (A1-E) match Sections 4 and 5.

4. The Architecture Diagram Explained, Piece by Piece

4.1 How to read the diagram

Visual element	What it means
Big dashed boxes (zones A-E)	Groups of parts with the same job: who asks, the front door, our services, OCI services, and dashboards.
Small solid boxes	Individual components. Labels match Section 5.
Blue (Zone A)	The people and systems that ask for machines.
Purple (Zone B)	The single front door - the Provisioning API.
Green boxes (Zone C, top row)	The main path every request follows: check, run, build, verify.
Grey boxes (Zone C, bottom row)	Shared services that every step uses: image catalog, state database, cleanup, identity.
Orange (Zone D)	The seven Oracle Cloud services our automation uses.
Thick orange arrow	Our services calling OCI through the OCI SDKs and the Terraform OCI provider.
Red side bar (Zone E)	Fleet dashboards - where the 45% result is measured.
Red dashed arrows	Data going to dashboards, and the feedback loop back to the pre-flight checks.

4.2 Zone A - Who asks for machines

- **A1 Engineers / Teams:** use a command-line tool, a web portal, or a REST call.
- **A2 CI/CD Pipelines:** create test environments automatically for code changes.
- **A3 Terraform Users:** describe infrastructure in code; their modules call the same rules.
- **Three blue arrows:** every request goes through the same front door. No side doors.

4.3 Zone B - Provisioning API (Java)

A wide box because everything passes through it. It checks **who** is asking, whether the request is **valid**, prevents **duplicates** with an idempotency key, applies **rate limits**, and returns a **request ID** the caller can use to track progress. The **purple arrow** passes the request to the pre-flight checks.

4.4 Zone C - Core services (running on OKE)

Top row - the main path (green arrows, in order):

- **C1 Pre-flight Checks (Go):** before creating anything - quota, capacity, free IPs, permissions, image fits the shape.
- **C2 Workflow Engine (Go):** runs the steps in order, retries with backoff, tries another AD or fault domain when capacity is short, and rolls back if it cannot finish.
- **C3 Terraform Runner:** runs reusable modules for network, VM, and disks: plan, check, apply. State is stored in Object Storage.
- **C4 Readiness Check (Go):** after creating - VM running, disks mounted, agent and SSH working. Only then is the request marked successful.

Bottom row - shared services (grey, used by every step):

- **C5 Image Lifecycle (Java):** the catalog of OS images and their states, so requests always get a valid, approved image.
- **C6 State DB + Events:** records every request, every step, and every error with its cause.
- **C7 Reconciler (Go):** runs on a timer, finds leftovers and drift, and cleans them up.
- **C8 Identity + Secrets:** services use workload identity and least-privilege IAM; secrets live in OCI Vault.

4.5 Zone D - OCI services

Box	What our automation uses it for
D1 Compute	Launch, stop, resize, and terminate VMs; shapes; images; instance configurations.
D2 VCN	Networks, subnets, network security groups, route tables, gateways.
D3 Block Volume	Boot volumes and data disks; attach and detach; backups.
D4 Object Storage	OS image files, Terraform state, logs, and reports.
D5 IAM	Compartments, policies, dynamic groups, and tags.
D6 OCIR	Container registry holding our services' container images.
D7 OKE	The Kubernetes cluster that runs our services.

4.6 Zone E and the feedback loop

The **fleet dashboards** show requests per day, success rate, failures by cause, breakdowns by region, AD, shape and team, time to ready, and orphan count. Red dashed arrows bring data from the readiness check (C4) and from OCI (Zone D). The **red dashed loop** at the bottom and left carries the top failure causes back to **C1** as new pre-flight checks and smarter retry rules.

5. Every Component Explained in Detail

Each component has four parts: **In simple terms**, **Detailed breakdown**, **Why it matters**, and an **Everyday analogy**. Several also show short example code or commands (grey boxes, simplified). Labels match the diagram.

A1 - Engineers and Teams

In simple terms

People who need machines for development, testing, or production.

Detailed breakdown

- **Ways to ask:** CLI tool, internal web portal, or direct REST call.
- **What they send:** shape, OS, size of disks, network (which subnet), team/compartment, tags.
- **What they get back:** a request ID immediately, and later the VM details (OCID, IP) or a clear error with its cause.

Why it matters

A simple, consistent experience instead of many manual console steps.

Everyday analogy: Staff filling in one order form at reception.

A2 - CI/CD Pipelines

In simple terms

Automated pipelines that need temporary machines to test code.

Detailed breakdown

- **Create** environments at the start of a test run and **destroy** them at the end.
- **High volume** - many requests at once, which used to hit API throttling.
- **Use the same API** with idempotency keys so re-running a pipeline does not create duplicates.

Why it matters

Pipelines were a large source of failures and leftovers before.

Everyday analogy: A training department booking temporary rooms every day.

A3 - Terraform Users

In simple terms

Teams that describe their infrastructure in Terraform code.

Detailed breakdown

- **Use shared modules** that already contain good defaults (tags, security rules, approved images).

- **Get the image ID** from the Image Lifecycle service instead of hard-coding it.
- **Remote state** in Object Storage with locking so two people do not change the same thing at once.

Example

```
module "app_vm" {
  source      = "git::https://.../modules/compute"
  shape      = "VM.Standard.E5.Flex"
  ocpus      = 2
  image_id   = data.external.approved_image.result.ocid
  subnet_id  = module.vcn.private_subnet_id
}
```

Why it matters

Everyone uses the same tested building blocks.

Everyday analogy: Using a standard floor-plan template instead of drawing from scratch.

B - Provisioning API (Java)

In simple terms

The single front door for every machine request.

Detailed breakdown

- **Built in Java** (for example with Spring Boot) using the OCI Java SDK; exposes a REST API described with OpenAPI.
- **Authentication:** checks the caller's identity and team.
- **Validation:** allowed shapes, sizes, regions, compartments, and required tags; clear error messages.
- **Idempotency key:** if the same request arrives twice (for example a pipeline retry), it returns the first request instead of creating a second VM. The same key is passed to OCI as a retry token.
- **Rate limits and queue:** smooths bursts so OCI APIs are not flooded (fewer 429 errors).
- **Asynchronous:** replies at once with a request ID (HTTP 202); callers poll or get a callback.
- **Writes** the request to the State DB (C6).

Example

```
LaunchInstanceRequest req = LaunchInstanceRequest.builder()
    .launchInstanceDetails(details)
    .opcRetryToken(requestId)    // same token on retry = no duplicate VM
    .build();
```

Why it matters

One place to enforce rules, prevent duplicates, and record everything.

Everyday analogy: A reception desk that checks your badge and gives you a ticket number.

C1 - Pre-flight Checks (Go)

In simple terms

Checks that the request can succeed BEFORE anything is created.

Detailed breakdown

- **Service limits and quotas:** enough cores, memory, and volumes left in that compartment and AD.
- **Capacity:** is that shape likely available in that AD right now (OCI compute capacity report)? If not, suggest or pick another AD or fault domain.
- **Image-shape fit:** the chosen image is approved and compatible with the chosen shape.
- **Network:** the subnet exists and has enough free IP addresses; NSGs exist.
- **Permissions:** the service identity is allowed to create these resources in that compartment.
- **Storage:** requested volume sizes and performance are within limits.
- **Runs checks in parallel** (Go goroutines) so it adds only seconds.
- **Fails fast with a clear reason** ("subnet has 3 free IPs, needs 5") instead of a half-created VM.

Example

```
oci limits resource-availability get --service-name compute \
  --limit-name standard-e5-core-count --compartment-id $C --availability-domain
$AD
oci compute image-shape-compatibility-entry list --image-id $IMAGE
```

Why it matters

The biggest single source of the 45% improvement: most failures were predictable.

Everyday analogy: Checking there is a free room, budget, and approval before ordering furniture.

C2 - Workflow Engine (Go)

In simple terms

Runs each provisioning step in order and handles problems along the way.

Detailed breakdown

- **State machine:** each request moves through clear states (validating, network, compute, storage, readiness, done/failed), saved in C6 so it survives restarts.
- **Waiters:** waits for OCI resources to reach the right state (for example instance RUNNING, attachment ATTACHED) with timeouts.
- **Smart retries:** retry only safe, temporary errors (429, 5xx, timeouts) with exponential backoff and jitter; never retry permanent errors (bad input, no permission).

- **Capacity fallback:** on "out of host capacity", try another fault domain or AD, or an allowed alternative shape, if the request permits.
- **Rollback (compensation):** if a later step fails for good, undo earlier steps (terminate VM, delete volume) so nothing is left behind.
- **Concurrency limits** per compartment and region to stay under API limits.
- **Classifies every error** into a cause for the dashboard.

Example

```
// simplified retry rule
if isTransient(err) && attempt < maxAttempts {
    sleep(backoff(attempt) + jitter())
    continue
}
return classify(err) // permanent: stop, roll back, report cause
```

Why it matters

Turns short, temporary problems into successes and stops leaving half-built machines.

Everyday analogy: A project manager who calls back a busy supplier, tries another warehouse, and cancels cleanly if needed.

C3 - Terraform Runner

In simple terms

Creates and changes infrastructure from reusable Terraform code.

Detailed breakdown

- **Modules:** vcn (network), compute (VM), block-volume (disks), iam (policies), oke (cluster).
- **Pinned versions** of Terraform and the OCI provider for repeatable results.
- **Plan -> check -> apply:** create a plan, run policy checks (for example required tags, no public IPs where forbidden), then apply.
- **Remote state** stored in Object Storage with locking, one state per environment.
- **Drift detection:** a scheduled plan that shows changes means someone changed resources by hand.
- **Outputs** (OCIDs, IPs) are returned to the workflow engine.

Example

```
terraform init -backend-config=env/prod.hcl
terraform plan -out=tfplan -detailed-exitcode
terraform apply tfplan
```

Why it matters

Everything is created the same way every time, and changes are reviewed.

Everyday analogy: Building from a standard blueprint instead of improvising.

C4 - Readiness Check (Go)

In simple terms

Confirms the machine really works before calling the request a success.

Detailed breakdown

- **Instance state** RUNNING.
- **Boot finished:** cloud-init completed without errors.
- **Agent reporting** to OCI monitoring.
- **Network:** has the expected private IP; SSH port reachable from the bastion or health checker.
- **Storage:** data volumes attached and mounted.
- **Timeout:** if not ready in time, the workflow retries or rolls back.
- **Records** time-to-ready for the dashboard.

Why it matters

A VM that exists but does not work is still a failure for the user. Catching it here gives honest numbers.

Everyday analogy: The final inspection before handing over the keys.

C5 - Image Lifecycle Service (Java)

In simple terms

The official catalog of OS images and their status.

Detailed breakdown

- **States:** candidate -> approved -> default -> deprecated -> deleted.
- **Import:** registers new images from Object Storage into Compute, with capability settings and shape compatibility.
- **Replication:** copies approved images to all needed regions.
- **Resolve by name:** callers ask for "oracle-linux-9 latest approved" and get the right OCID for their region and shape - no hard-coded, deleted, or incompatible image IDs.
- **Deprecation window:** warns users before an image is removed; blocks new launches from deleted images.
- **Integration:** the API, pre-flight checks, and Terraform modules all ask this service for image IDs.

Why it matters

"Image not found" and "image not compatible with shape" failures almost disappeared.

Everyday analogy: A product catalog that only shows items in stock.

C6 - State DB and Events

In simple terms

The memory of the system - every request and every step.

Detailed breakdown

- **Request record:** who, what, when, current state.
- **Step records:** each step's start, end, result.
- **Error cause:** a standard category (quota, capacity, IAM, IP, image, throttling, attach, other) plus the OCI error code and message.
- **Events** published for dashboards and notifications.
- **Audit trail** for troubleshooting and compliance.

Why it matters

Makes restarts safe and gives the dashboards accurate data.

Everyday analogy: The order book at reception.

C7 - Reconciler and Cleanup (Go)

In simple terms

A background job that finds and fixes leftovers and mismatches.

Detailed breakdown

- **Compares** what the State DB says should exist with what really exists in OCI (using tags like managed-by and request-id).
- **Orphans:** unattached volumes, stopped VMs from failed runs, reserved IPs - cleaned up after a safety period.
- **Stuck requests:** resumes or fails them cleanly.
- **Drift:** reports or corrects resources changed outside the system.
- **Frees quota and IPs**, which directly prevents future quota and IP failures.

Why it matters

Leftovers were silently causing new failures by using up quota and IPs.

Everyday analogy: The nightly cleaner who clears abandoned boxes from hallways.

C8 - Identity and Secrets

In simple terms

How our services safely prove who they are to OCI.

Detailed breakdown

- **No stored passwords:** services use OKE workload identity (or instance principals) to call OCI.

- **Dynamic groups and policies** give each service only the permissions it needs, in specific compartments.
- **OCI Vault** holds any secrets (for example database passwords).
- **Tag-based policies** limit the services to resources they manage.

Why it matters

Fewer permission errors and much lower security risk.

Everyday analogy: Staff badges that open only the doors they need.

D1 - OCI Compute

In simple terms

Oracle's virtual machine service.

Detailed breakdown

- **Launch** instances with a shape, image, subnet, and boot volume.
- **Flexible shapes** (choose OCPUs and memory).
- **Instance configurations and pools** for groups of identical VMs.
- **Stop, start, resize, terminate.**
- **Capacity errors** happen when an AD has no free hardware of that type - handled by pre-flight and fallback.

Why it matters

The core resource being provisioned.

Everyday analogy: The rooms in the building.

D2 - VCN (Virtual Cloud Network)

In simple terms

The private network for the VMs.

Detailed breakdown

- **Subnets** with IP ranges; running out of IPs caused failures.
- **Network security groups and security lists** (firewall rules).
- **Route tables and gateways** (internet, NAT, service gateway).
- **VNICs** attach VMs to subnets.

Why it matters

No network means no working VM.

Everyday analogy: Corridors and phone lines.

D3 - Block Volume

In simple terms

Disks for VMs.

Detailed breakdown

- **Boot volume** created from the image.
- **Data volumes** created and attached (paravirtualized or iSCSI).
- **Performance levels** and sizes.
- **Backups and policies.**
- **Attach timeouts** handled by waiters and retries.

Why it matters

Storage problems were a common cause of "VM created but not usable".

Everyday analogy: Filing cabinets.

D4 - Object Storage

In simple terms

File storage used by the automation.

Detailed breakdown

- **OS image files** (for example QCOW2) to import.
- **Terraform remote state** files.
- **Logs and reports** from runs.
- **Access controlled** by IAM; versioning on state buckets.

Why it matters

Safe, shared storage for everything the automation needs to remember.

Everyday analogy: The records room.

D5 - IAM

In simple terms

Who is allowed to do what.

Detailed breakdown

- **Compartments** to separate teams and environments.
- **Policies** in plain language ("allow dynamic-group X to manage instances in compartment Y").
- **Dynamic groups** for services.
- **Tags** for ownership and cost; tag defaults applied automatically.

Why it matters

Wrong permissions were a top failure cause; standard policies fixed most of them.

Everyday analogy: The key-card system.

D6 - OCIR (Container Registry)

In simple terms

Stores the container images of our Java and Go services.

Detailed breakdown

- **CI builds** each service into a container image and pushes it to OCIR with a version tag.
- **Image scanning** for vulnerabilities.
- **OKE pulls** the images to run the services.
- **Access** limited by IAM.

Why it matters

Reliable, versioned delivery of our own software.

Everyday analogy: The warehouse for our office equipment.

D7 - OKE (Kubernetes Engine)

In simple terms

The Kubernetes cluster that runs our services.

Detailed breakdown

- **Runs** the API, pre-flight, workflow, readiness, image, and reconciler services as containers.
- **Multiple replicas** for high availability; automatic restarts.
- **Rolling updates** for new versions.
- **Workload identity** lets pods call OCI securely.
- **Scales** with request volume.

Why it matters

The automation itself must be highly available - if it is down, nobody can provision.

Everyday analogy: The building that houses reception and the back office.

E - Fleet Dashboards

In simple terms

Live charts that show how provisioning is going - and prove the 45%.

Detailed breakdown

- **Requests per day** and **success rate**.

- **Failures by cause** (the categories from C6).
- **Breakdown** by region, AD, shape, image, and team.
- **Time to ready** (p50, p95).
- **Retries, fallbacks, and rollbacks** counts.
- **Orphan count** from the reconciler.
- **Alerts** when success rate drops or a cause spikes.

Why it matters

Shows where to improve and proves the result with real numbers.

Everyday analogy: The notice board showing orders completed and why others failed.

6. The Complete Workflow, Step by Step

This follows one request for a new VM from the moment it is sent until the VM is handed over - and later retired.

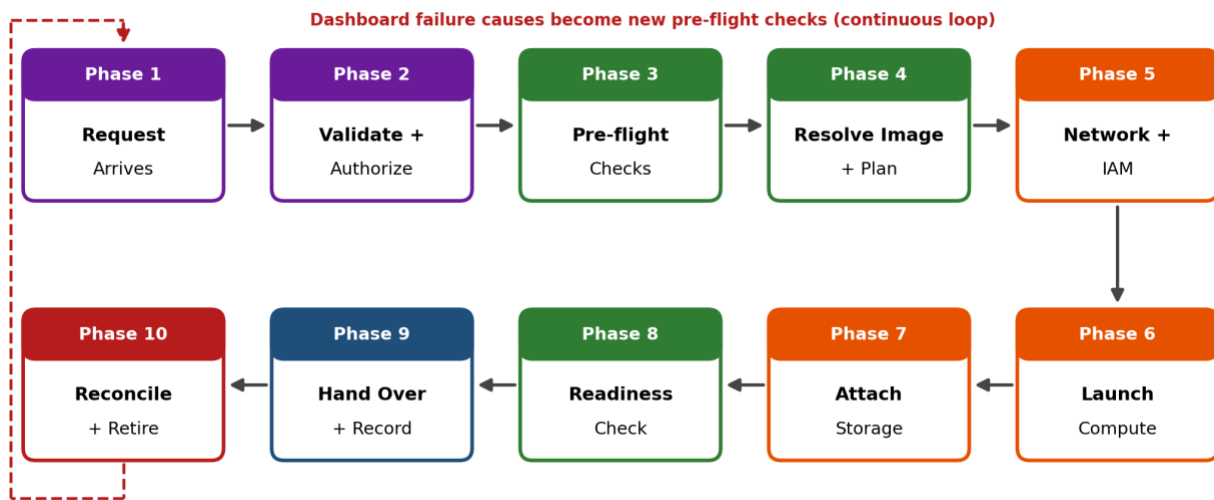


Figure 2 - The 10 phases. Purple = front door, green = checks, orange = creating resources, blue = hand-over, red = ongoing care.

Phase 1 - Request arrives

Step	What happens (detail)	In simple words
1.1	Caller sends a request (CLI, portal, pipeline, or Terraform) with shape, OS, disks, subnet, team, and an idempotency key.	Fill in the order form.
1.2	API replies immediately with a request ID (HTTP 202).	Get a ticket number.

Phase 2 - Validate and authorize

Step	What happens (detail)	In simple words
2.1	Check caller identity and team permissions.	Check the badge.
2.2	Validate fields: allowed shape, size, region, compartment, required tags.	Check the form is complete.
2.3	If the idempotency key was seen before, return the existing request.	No duplicate orders.
2.4	Save request in State DB; state = VALIDATING.	Write it in the order book.

Phase 3 - Pre-flight checks

Step	What happens (detail)	In simple words
3.1	In parallel: quota, capacity, subnet IPs, IAM permission, volume limits.	Is there room, budget, approval?
3.2	Any hard failure: stop with a clear cause; nothing is created.	Say no early and clearly.
3.3	Capacity low in the chosen AD: pick an allowed alternative AD or fault domain.	Try another floor.

Phase 4 - Resolve image and plan

Step	What happens (detail)	In simple words
4.1	Ask the Image Lifecycle service for the approved image OCID for this OS, region, and shape.	Pick a model that is in stock.
4.2	Terraform plan for network, VM, and disks; policy checks on the plan.	Draw up the work order and check it.

Phase 5 - Network and IAM

Step	What happens (detail)	In simple words
5.1	Create or reuse the VCN pieces (subnet, NSG rules) via Terraform modules.	Prepare the corridor and phone line.
5.2	Apply required tags and any per-request policy settings.	Label everything with the owner.

Phase 6 - Launch compute

Step	What happens (detail)	In simple words
6.1	Launch the instance with shape, image, subnet, and a retry token equal to the request ID.	Set up the room.
6.2	Temporary errors (429, 5xx): retry with backoff and jitter.	Supplier busy - call back later.
6.3	Out of capacity: try allowed fallback AD, fault domain, or shape.	Try another floor.
6.4	Wait until the instance is RUNNING (with timeout).	Wait for the room to be ready.

Phase 7 - Attach storage

Step	What happens (detail)	In simple words
7.1	Create data volumes; attach them (paravirtualized or iSCSI).	Deliver the filing cabinets.
7.2	Wait for ATTACHED; retry on timeout; roll back if it cannot attach.	Make sure they are really in the room.

Phase 8 - Readiness check

Step	What happens (detail)	In simple words
8.1	Check cloud-init finished, agent reporting, IP present, SSH reachable, disks mounted.	Final inspection.
8.2	If not ready in time: retry or roll back everything created for this request.	Take it all back cleanly if it does not work.

Phase 9 - Hand over and record

Step	What happens (detail)	In simple words
9.1	State = READY; return OCID, IP, and details to the caller.	Hand over the keys.
9.2	Record time-to-ready, retries, fallbacks, and outcome; send events to dashboards.	Update the notice board.

Phase 10 - Reconcile and retire

Step	What happens (detail)	In simple words
10.1	Reconciler runs on a timer: finds orphans, drift, and stuck requests; cleans or fixes them.	Nightly cleaning.
10.2	Image lifecycle deprecates old images; users are warned; new launches use the new default.	Retire old models.
10.3	Terminate VMs when no longer needed (pipelines do this automatically).	Hand back the room.
10.4	Weekly review of top failure causes on the dashboard; add new pre-flight checks or retry rules.	Fix the biggest problem next.

7. VM and Image Lifecycle States

"Integrating VM and image lifecycle" means both have clear states, and they are linked: a VM can only be created from an image in a usable state.

7.1 VM request states

State	Meaning	Next
REQUESTED	Request received and saved.	VALIDATING
VALIDATING	Identity, fields, idempotency.	PREFLIGHT or REJECTED
PREFLIGHT	Quota, capacity, IPs, IAM, image fit.	PROVISIONING or REJECTED
PROVISIONING	Network, compute, storage being created.	VERIFYING or ROLLING_BACK
VERIFYING	Readiness checks.	READY or ROLLING_BACK
READY	Handed over and in use.	STOPPED or TERMINATING
ROLLING_BACK	Undoing partial work after a permanent failure.	FAILED
REJECTED / FAILED	Did not succeed; cause recorded.	(end)
TERMINATING / TERMINATED	Being removed / removed.	(end)

7.2 Image states

State	Meaning	Can new VMs use it?
CANDIDATE	Imported and being tested.	Only test requests
APPROVED	Passed all gates; replicated to regions.	Yes, if asked for by version
DEFAULT	The version given to "latest" requests.	Yes
DEPRECATED	Older version; users warned.	No new launches (existing VMs keep running)
DELETED	Removed from Compute and catalog.	No

8. Where the 45% Came From, and How It Was Measured

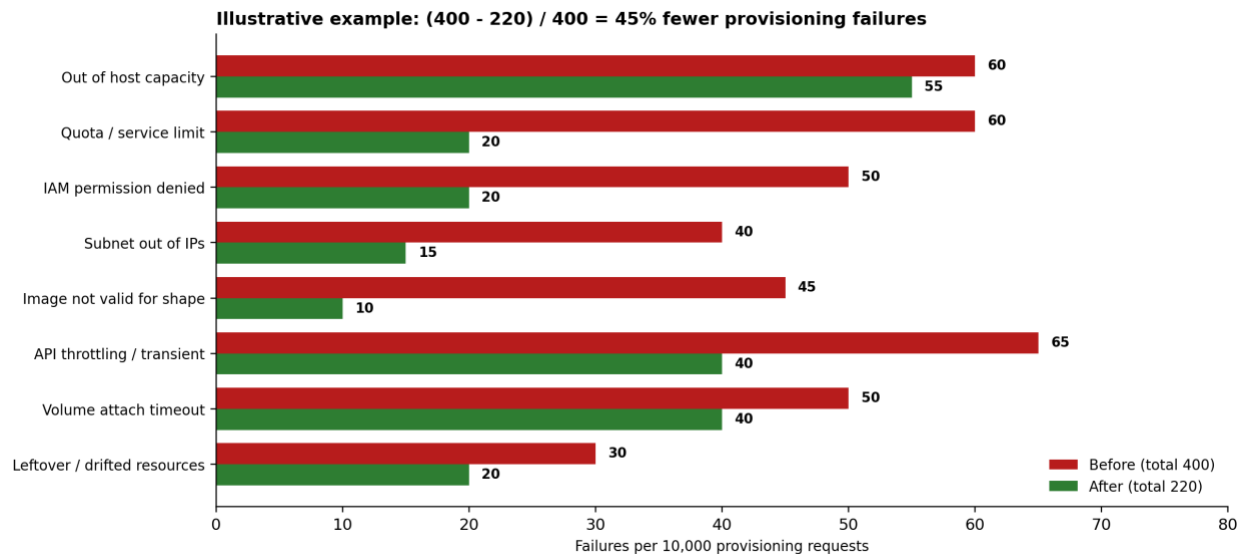


Figure 3 - Illustrative only. Real numbers come from your fleet dashboards.

8.1 Each failure cause and what fixed it

Failure cause	What went wrong before	What fixed it
Quota / service limit	Found only when the create call failed.	Pre-flight limit and quota checks; reconciler frees leftovers.
IAM permission denied	Inconsistent hand-made policies.	Standard policies, dynamic groups, workload identity; pre-flight permission check.
Subnet out of IPs	Discovered mid-launch.	Pre-flight free-IP check; cleanup of orphaned VNICs and IPs.
Image not valid for shape	Hard-coded, deleted, or incompatible image IDs.	Image Lifecycle service resolves the right image; compatibility check.
API throttling / transient	Bursts from pipelines; no smart retry.	Rate limiting and queue in the API; backoff with jitter; idempotent retries.
Out of host capacity	One AD only; failed immediately.	Capacity check and allowed AD / fault-domain / shape fallback. (Partly outside our control, so smaller drop.)
Volume attach timeout	No waiter; declared failed too early.	Proper waiters, retries, and readiness check.
Leftover / drifted resources	Half-created resources blocked later requests.	Rollback in the workflow; reconciler cleanup; drift detection.

8.2 How it was measured

- **Every request** is recorded with its outcome and, on failure, a standard cause category.
- **Failure rate** = failed requests / total requests, shown per day and week on the fleet dashboard.
- **Compare equal periods** before and after the rollout (for example three months each).
- **Use rates, not counts**, so growth in request volume does not hide or exaggerate the change.
- **Formula:** reduction = (old rate - new rate) / old rate. Example: 4.0% -> 2.2% = 45% fewer failures.
- **Readiness counts:** a VM that exists but fails readiness is counted as a failure, so the number is honest.

9. Safeguards and a 60-Second Explanation

9.1 Built-in safeguards

- **One front door** with validation and authentication.
- **Idempotency** everywhere - no duplicate VMs on retry.
- **Check before create** - most failures stopped before they cost anything.
- **Retry only what is safe** to retry.
- **Roll back** partial work on permanent failure.
- **Readiness check** before success.
- **Reconciler** removes leftovers and detects drift.
- **Least-privilege identity**, no stored passwords.
- **Highly available services** on OKE, delivered through OCIR.
- **Dashboards and alerts** on success rate and causes.

9.2 The 60-second explanation (for interviews)

"Provisioning VMs on OCI failed too often - quota, capacity, IAM, subnet IPs, wrong images, throttling, and leftovers from earlier failures. I built a single Java provisioning API with validation and idempotency keys, and Go services for pre-flight checks, a workflow engine, readiness checks, and a reconciler, plus shared Terraform modules with remote state in Object Storage. Pre-flight checks verified limits, capacity, free IPs, permissions, and image-shape compatibility before creating anything. The workflow retried only transient errors with backoff and jitter, fell back to other ADs or fault domains on capacity errors, and rolled back cleanly. A Java image lifecycle service made sure every launch used an approved, compatible image. Everything ran on OKE from images in OCIR, using workload identity and least-privilege IAM. Fleet dashboards showed a 45% drop in the provisioning failure rate."